Reproducibility Engineering
Summer Term 2026
Chair of Scalable Database Systems
Prof. Dr.-Ing. Stefanie Scherzinger, Stefan Klessinger, Christoph Köhnen

UNIVERSITY OF PASSAU
Faculty of Computer Science and Mathematics

Exercise Sheet 11

Lab Sessions: July 16/17, 2026

In this exercise sheet, we follow one small experiment through a reproducible workflow. Our running example is a classic kitchen-science experiment: how high does Cola erupt when Mentos are dropped into the bottle? We first package the experiment as a lean container using a *multi-stage build*, then run it on a *remote* server the way we would for a real, long-running experiment, and finally store and inspect its results in *HDF5* format.

1 Preparation

For this exercise, we provide files in the RepEng FIMGit Repository (directory `LabSession11/`). Update your local repository that you cloned in the first lab session:

1. Open a terminal in your local repository.
2. Pull the latest changes using `git pull`.

All tasks use Docker, which you have already installed from the previous lab sessions.

2 Multi-Stage Builds

Building software often requires many more dependencies than just running it. Compiling our experiment requires a full C toolchain; running the compiled program requires almost nothing. A *multi-stage build* exploits this: one stage builds the artifact, and a second, minimal stage copies only what is needed at run time. The toolchain stays behind in the discarded build stage, so the final image is small, and is thus faster to transfer and cheaper to store.

We provide the experiment as a C program, `multistage/mentos.c`, that prints the measurements as CSV. The directory also contains a naive Dockerfile using a single stage, `Dockerfile.singlestage`.

2.1 The Single-Stage Image

Enter the directory `multistage/` and build the single-stage image, then check its size:

Terminal Session bash

```
user@host$ cd LabSession11/multistage
user@host$ docker build -f Dockerfile.singlestage -t mentos-singlestage .
user@host$ docker images mentos-singlestage
```

The image is based on `gcc:14`, so it carries the entire compiler toolchain.

2.2 The Multi-Stage Image

Now write a mulit-stage[1] `Dockerfile` with two stages:

1. A **build stage** based on `gcc:14` that copies in `mentos.c` and compiles it to a self-contained (statically linked) binary.

   Hint: Reuse the `gcc` command from `Dockerfile.singlestage`.

2. A **runtime stage** based on `scratch` (the empty image) that copies only the compiled binary from the build stage and runs it as its entrypoint.

   Hint: Use `COPY --from=<...>` to copy from another build stage.

Build it and compare the two image sizes:

Terminal Session bash

```
user@host$ docker build -t mentos-multistage .
user@host$ docker images | grep mentos
```

Finally, test the multi-stage image:

Terminal Session bash

```
user@host$ docker run --rm mentos-multistage > measurements.csv
```

By how much did the image shrink in size, and *why*?

[1] https://docs.docker.com/build/building/multi-stage/

3 Working with Remote Containers

Experiments are often run on remote machines, such as a lab server or a cluster, so we can use more powerful hardware and free up our own computer. However, a remote machine may only provide an SSH login and might not even support Docker. In such situations, a good practice is to build a self-contained *experiment execution package* in a controlled environment, ship it to the remote machine, run it there, and bring the results back for analysis in a controlled environment, together with a record of the machine where they were generated.

In this exercise, we simulate such a remote server with a local container: it runs an SSH server, and we connect to it over the network as we would to a real remote machine.

3.1 Setting up the "Remote" Server

Enter `remote/`, build the server image from `Dockerfile.remote`, and start it in the background. The container's SSH port (22) is published on your host as port 2222:

Terminal Session bash

```
user@host$ cd LabSession11/remote
user@host$ docker build  -f Dockerfile.remote -t lab11-remote .
user@host$ docker run --rm -d -p 2222:22 --name lab11-remote lab11-remote
```

The server is now running and waiting for connections. We will not connect to it directly from the host; instead, we connect from the "local" build container in the next task.

3.2 Building and Shipping the Experiment Execution Package

We provide the experiment as a C program with built-in pauses that mimic a long-running experiment. We compile it inside a build container to a self-contained (statically linked) binary, so it runs on the remote machine without Docker. In the same container, we package it with the dispatcher `dispatch.sh` and copy it to the remote machine.

The `Dockerfile.builder` in `remote/` builds this container: it compiles `experiment.c` and carries the tools needed to package and ship the result. Build it, then start it with `--network host` so it can reach the "remote" server at `localhost:2222`:

Terminal Session bash

```
user@host$ docker build -f Dockerfile.builder -t lab11-experiment-builder .
user@host$ docker run --rm -it --network host lab11-experiment-builder
```

Inside the build container, bundle the binary and the dispatcher into `experiment.tar.gz` and copy it to the remote server (the password is `repro`):

Terminal Session bash

```
repro@builder$ tar -czf experiment.tar.gz package
repro@builder$ scp -P 2222 experiment.tar.gz repro@localhost:~/
```

3.3 Running an Experiment that Survives Disconnects

Now connect to the "remote" server from the build container and run the experiment there. It takes a while to run; if we simply ran it and then closed the connection, the process would be killed. We therefore run it inside `tmux`, a terminal multiplexer that keeps a session alive independently of our connection.

1. Connect to the remote server, unpack the package, start a `tmux` session, and launch the dispatcher inside it:

   Terminal Session bash

   ```
   repro@builder$ ssh -p 2222 repro@localhost
   repro@remote$ tar -xzf experiment.tar.gz
   repro@remote$ cd package
   repro@remote$ chmod +x mentos dispatch.sh
   repro@remote$ tmux new -s mentos
   repro@remote$ ./dispatch.sh
   ```

2. While the script is running, *detach* from the session by pressing `Ctrl-b` and then `d`, then log out of the SSH session with `exit`.

3. Reconnect and *reattach* to the session. Is the experiment still running (or finished)?

   Terminal Session bash

   ```
   repro@builder$ ssh -p 2222 repro@localhost
   repro@remote$ tmux a
   ```

   Once the experiment is finished, detach from the `tmux` session again by pressing `Ctrl-b` and then `d`.

3.4 Inspecting the Recorded Environment

The information in which execution environment results were produced is important *provenance data*. The dispatcher therefore records some characteristics of the remote machine into a folder `out/config/`, alongside the measurements. Inspect what it collected and explain what the entries tell us:

Terminal Session bash

```
repro@remote$ cd package/out/config
repro@remote$ ls
repro@remote$ cat hostname os-release
```

Which of these would you most want to record for a real experiment, and why?

3.5 Collecting the Results and Analyzing Them Locally

Once the experiment has finished, bundle the measurements together with the recorded configuration and copy that archive back to your local build container with `scp`:

Terminal Session bash

```
repro@remote$ tar -czf results.tar.gz out
repro@remote$ exit
repro@builder$ scp -P 2222 repro@localhost:~/package/results.tar.gz .
```

Note that `ssh` uses lowercase `-p` for the port, while `scp` uses uppercase `-P`. Back in the build container, you analyze the results in the same controlled (containerized) environment as the rest of the workflow – so the analysis, too, runs reproducibly.

Unpack the archive and run the provided `plot.py`, which reads `out/measurements.csv` and writes a chart about the experiment to `out/measurements.png`:

Terminal Session bash

```
repro@builder$ tar -xzf results.tar.gz
repro@builder$ python3 plot.py
```

Finally, write a short `README.md` that describes the results, and include a description of the machine the experiment ran on (taken from `out/config/`). During the lab session, you may show the README to your tutor for feedback.

When you are done with this exercise, stop the server:

Terminal Session bash

```
user@host$ docker stop lab11-remote
```

3.6 Optional: Connecting from an IDE

Many editors can work directly on a remote machine over SSH, for example the Remote-SSH extension for VS Code or JetBrains Gateway. Configure your editor to connect to `repro@localhost` on port 2222 and open `dispatch.sh` remotely.

4 Storing and Inspecting Experimental Data in HDF5

HDF5 is a compact, self-describing container format for large, hierarchical datasets. A file is organized into *groups* (like directories) and *datasets* (n-dimensional arrays), both of which may carry *attributes* (metadata). In this part, we take the measurements from our experiment, store them in an HDF5 file, and then inspect the result.

The results are provided as a CSV file, `measurements.csv`, with the columns `flavor`, `cola`, `mentos`, and `height_cm` – the same data the multi-stage image produces and the remote experiment writes.

4.1 Preparation: HDF5 Tools and Library

The `Dockerfile` in `LabSession11/hdf5/` provides the HDF5 command-line tools `h5ls` and `h5dump`, the Python library `h5py`, and `measurements.csv`. Build the image and run the container:

Terminal Session bash

```
user@host$ cd LabSession11/hdf5
user@host$ docker build -t lab11-hdf5 .
user@host$ docker run -it lab11-hdf5
```

The accompanying *HDF5 cheatsheet* (available in StudIP) lists the `h5py` functions needed for the next task.

4.2 Writing an HDF5 File

Write a Python script named `store.py` that reads `measurements.csv` and stores its contents in a new HDF5 file `measurements.h5`. Use a structure that mirrors the experiment:

- a top-level group `meas`;
- a dataset with path `meas/<flavor>/<cola>` for every combination of flavor and cola;
- each dataset is a two-dimensional array with rows `[mentos, height_cm]`;
- add an attribute `columns` with value `["mentos", "height_cm"]` to each dataset, documenting the two columns.

Hint: The cheatsheet lists every function you need. These are `h5py.File`, `create_group`, `create_dataset`, and assigning to `x.attrs[...]`. You may read the CSV with the `csv` module or with `pandas`.

After implementing it, run your script:

Terminal Session bash

```
user@container$ python3 store.py
```

4.3 Inspecting the File You Created

Now inspect `measurements.h5` to confirm that it has the structure you intended, using two tools from `hdf5-tools`.

(a) List the object tree (groups and datasets) recursively:

Terminal Session bash

```
user@container$ h5ls -r measurements.h5
```

Does the hierarchy match the requested group structure?

(b) Dump one dataset in full, including its values and attributes:

Terminal Session bash

```
user@container$ h5dump -d /meas/fruit/cola measurements.h5
```

What does the `columns` attribute contain, and what are the shape and datatype of the dataset?

(c) For a large file, printing every value is impractical. Print only the structure and metadata (the *header*), without the data values:

Terminal Session bash

```
user@container$ h5dump -H measurements.h5
```

What does `h5dump` show here that `h5ls -r` does not?

4.4 Reading the Data back in Python

Now write a Python script called `read.py` that reads the eruption heights you stored in the HDF5 file. Refer to the HDF5 cheatsheet for the necessary functions.

1. Open `measurements.h5` for reading and access the dataset `meas/mint/diet`.
2. Print its `columns` attribute and the stored array.

After implementing it, run your script:

Terminal Session bash

```
user@container$ python3 read.py
```

5 Reproducible Workflows (Antwort-Wahl-Verfahren)

In this task, each question has exactly one correct answer. Mark the correct answer.

(a) The two Dockerfiles below build the same program `mentos.c`. The first one uses a single build stage, the second one a multi-stage build:

Dockerfile (singles-tage)

```
FROM gcc:14
WORKDIR /src
COPY mentos.c .
RUN gcc -O2 -static -o mentos mentos.c -lm
ENTRYPOINT ["/src/mentos"]
```

Dockerfile (multi-stage)

```
FROM gcc:14 AS builder
WORKDIR /src
COPY mentos.c .
RUN gcc -O2 -static -o mentos mentos.c -lm

FROM scratch
COPY --from=builder /src/mentos /mentos
ENTRYPOINT ["/mentos"]
```

How many of the following statements about the two resulting images are **true**?

(i) The image built from the multi-stage `Dockerfile` is smaller.

(ii) In the single-stage image, you can call `gcc` to recompile the program, in the multi-stage image, you cannot.

(iii) The command `./mentos` can only be run in the single-stage image.

(iv) The `mentos` binary is bitwise identical in both images.

(v) Both images contain the source file `mentos.c`.

○ 0 ○ 1 ○ 2 ○ 3 ○ 4 ○ 5

(b) You run `h5ls -r results.h5` and get the following output:

```
h5ls -r results.h5
/                        Group
/experiment              Group
/experiment/run1         Dataset {120, 2}
/experiment/run2         Dataset {120, 2}
/experiment/run3         Dataset {120, 2}
```

How many of the following statements about `results.h5` are **true**?

(i) `results.h5` contains three datasets.

(ii) Each of `run1`, `run2`, and `run3` stores $120 \times 2 = 240$ values.

(iii) `/experiment` is a group, and the three datasets are nested inside it.

(iv) Each run stores 120 values, and 2 of them are missing (`NaN`).

(v) The runs hold floating-point numbers with 2 decimal places.

○ 0 ○ 1 ○ 2 ○ 3 ○ 4 ○ 5

(c) You must store 50 GB of numerical measurements from many experiment runs and later read arbitrary slices of it efficiently. How many of the following statements about storing this data as HDF5 and/or JSON are **true**?

(i) JSON files contain an index, so a reader can jump directly to a specific run without parsing the rest.

(ii) HDF5 lets you read a single run (a slice) without loading the whole file.

(iii) JSON is stored as raw text, so for 50GB of numerical data, the file is typically larger than an HDF5 file containing the same data.

(iv) HDF5 can also store images, such as charts.

(v) JSON can also store images, such as charts.

○ 0 ○ 1 ○ 2 ○ 3 ○ 4 ○ 5